

Detailný rozbor projektu — Spracovanie vstupných údajov v Kubernetes

exam-horvath

2026-05-14

0. Mapa projektu

sablona-cvicne-zadanie/

src/	# PHP aplikácia
index.php	# hlavná stránka + spracovanie CSV
health.php	# livenessProbe endpoint
ready.php	# readinessProbe endpoint
k8s/	# všetky Kubernetes manifesty
namespace.yaml	# 1. Namespace
configmap.yaml	# 2. ConfigMap
pv.yaml	# 3a. PersistentVolume
pvc.yaml	# 3b. PersistentVolumeClaim
deployment-upload-init.yaml	# 4+5. InitContainer + Deployment
service.yaml	# 6. Service NodePort

Pokrytie zadania (mapovanie na hodnotenie):

Požiadavka zo zadania	Súbor	Body
Namespace + štruktúra	namespace.yaml	5
ConfigMap s vplyvom na správanie	configmap.yaml	10
Funkčný InitContainer	deployment-upload-init.yaml	10
PVC a zdieľaný filesystem	pv.yaml + pvc.yaml	10
Deployment a mountovanie volume	deployment-upload-init.yaml	10
Service NodePort	service.yaml	10
README + popis spracovania	README.md	5
Bonus: readinessProbe + livenessProbe	Deployment	+

1. k8s/namespace.yaml — Namespace

```
apiVersion: v1
kind: Namespace
metadata:
  name: exam-horvath
```

Pole-po-poli

apiVersion: v1

- **Povinné? ÁNO** — bez apiVersion kubectl odmietne manifest s chybou “no kind specified”.
- **Čo sa píše?** Verzia Kubernetes API, ktorá pozná dané kind. Namespace je *core* (jadrový) objekt, takže API group je prázdna a verzia je v1.
- **Iné možnosti?** Žiadne — Namespace existuje len v v1.

kind: Namespace

- **Povinné? ÁNO** — určuje typ objektu, ktorý sa má vytvoriť.
- **Čo sa píše?** Presný (case-sensitive) názov resource typu. namespace malými písmenami nefunguje.
- **Účel:** logické oddelenie zdrojov v klastri (zvlášť pre cvičné zadanie podľa pokynu “namespace exam-*{vaso_meno}*”).

metadata:

- **Povinné? ÁNO** — musí obsahovať aspoň name.
- **Čo sa píše?** Metadáta objektu (názov, labely, anotácie, ...).

metadata.name: exam-horvath

- **Povinné? ÁNO** — bez mena nebude vedieť API server, ako objekt vytvoriť.
- **Pravidlá:** DNS-1123 label — max. 63 znakov, len a–z, 0–9, -; musí začínať a končiť písmenom/číslom.
- **Tu:** dodržiava zadanie exam-*{vaso_meno}* → exam-horvath.
- **Pozor:** namespace sa za behu **nedá premenovať** — name je immutable. Pri zmene treba zmazať a vytvoriť nový.

Čo by sa dalo pridať (nie povinné)

- metadata.labels: — napr. purpose: exam na filtrovanie (kubectl get ns -l purpose=exam).
- metadata.annotations: — neselektívne metadata (popis, autor).

2. k8s/configmap.yaml — ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: exam-horvath
data:
  APP_NAME: "Exam HorvathVK"
  LOG_LEVEL: "INFO"
  PROCESS_MODE: "prepend-marker"
  INPUT_DIR:  "/var/www/html/data/input"
  OUTPUT_DIR: "/var/www/html/data/output"

  WELCOME_TEXT: |
    Vitajte v aplikácii!
    Tento súbor pripravil InitContainer pri štarte podu.
    Namespace: exam-horvath
    Aplikácia: PHP úloha

  SAMPLE_CSV_1: |
    name,age,city
    Alice,30,Bratislava
    Bob,25,Košice
    Cara,40,Žilina

  SAMPLE_CSV_2: |
    product,price,stock
    Pero,1.50,100
    Zošit,2.00,50
    Kniha,12.50,15
```

Pole-po-poli

apiVersion: v1

- **Povinné? ÁNO.** ConfigMap je core objekt → v1.

kind: ConfigMap

- **Povinné? ÁNO.** Presná hodnota.

metadata.name: app-config

- **Povinné? ÁNO.** Tento názov používa Deployment pri referencovaní (configMapRef.name: app-config a vo volumes).

metadata.namespace: exam-horvath

- **Povinné? Technicky NIE, ale prakticky veľmi odporúčané.** Bez namespace: sa ConfigMap vytvorí v *aktuálnom* namespace (podľa `kubectl config current-context`). Ak by sme zabudli, ConfigMap by vznikol v default a Deployment v exam-horvath ho **nenájde** (ConfigMap musí byť v rovnakom namespace ako Pod).
- **Pravidlo:** explicitne uvádzaj namespace v každom manifeste — uchráni ťa pred náhodným nasadením do default.

data: — vlastné údaje ConfigMapu

- **Povinné? NIE striktne** — ConfigMap môže mať aj prázdne data alebo používať binaryData. Ale prázdny ConfigMap je zbytočný.
- **Čo sa píše?** Mapa kľúč: hodnota. Hodnoty **musia byť string** (UTF-8). Pre čísla/booleany sa **musia obaliť do úvodzoviek** ("10", nie 10) — inak YAML parser pošle do API integer a API vráti chybu.
- **Limit:** celkový obsah ConfigMapu je obmedzený na **1 MiB** (limit etcd kľúča).

Jednotlivé kľúče v tomto súbore

Kľúč	Typ použitia v aplikácii
APP_NAME	Cez envFrom → getenv('APP_NAME') v index.php. Zobrazí sa v UI.
LOG_LEVEL	Env var (demonštruje vplyv ConfigMapu na správanie — log úroveň).
PROCESS_MODE	Env var (režim spracovania = prepend-marker).
INPUT_DIR	Cesta, kde aplikácia hľadá CSV. Aplikácia ju číta cez getenv('INPUT_DIR').
OUTPUT_DIR	Cesta, kam aplikácia píše transformovaný CSV.
WELCOME_TEXT	Cez volumes.configMap.items namountovaný ako /cfg/welcome.txt, InitContainer ho skopíruje do PVC.
SAMPLE_CSV_1/2	Sample CSV súbory — InitContainer ich kopíruje do input/.

YAML block scalar | (literal)

- Znak | znamená “zachovaj nové riadky tak ako sú”. Hodnota končí na zmene odsadenia.
- Alternatíva > by skladala riadky do jedného odseku — pre CSV nepoužiteľné.
- | - ukrojí koncový newline, |+ zachová všetky koncové newliny.

Ako sa ConfigMap konzumuje (mimo tohto súboru, ale relevantné)

1. **Ako env vars** — envFrom.configMapRef v Deployment vloží všetky kľúče ako env premenné.
2. **Ako súbory** — volumes.configMap.items namountuje vybrané kľúče ako súbory.

Čo by sa dalo pridať

- binaryData: — pre binárne súbory (base64).
- immutable: true — zakáže zmeny, optimalizuje výkon API serveru (od 1.19+).

3a. k8s/pv.yaml — PersistentVolume

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: exam-pv
spec:
  storageClassName: manual
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/mnt/data/exam-horvath"
```

Pole-po-poli

`apiVersion: v1 + kind: PersistentVolume`

- **Povinné. Core objekt.**

`metadata.name: exam-pv`

- **Povinné.** Cluster-wide názov — PV je **non-namespaced** objekt (`kubectl get pv` bez `-n`).
- Preto tu **nie je** namespace: — bolo by ignorované.

`spec: — definícia objemu`

- **Povinné.**

`spec.storageClassName: manual`

- **Povinné? NIE striktné**, ale ak ho vynecháš, K8s sa pokúsi automaticky priradiť default StorageClass — to môže pokaziť párovanie s PVC.
- **Účel:** “lepidlo” medzi PV a PVC. PVC sa naviaže iba na PV s **rovnakou** storageClassName.
- **Hodnota manual:** dohovoréné meno pre static-provisioned PV (nikde sa nepoužíva dynamický provisioner). Mohli by sme dať aj "" (prázdny string) — vtedy K8s nepoužije žiaden default.

`spec.capacity.storage: 1Gi`

- **Povinné.** Veľkosť úložiska.
- **Jednotky:** Ki/Mi/Gi/Ti (binary) alebo K/M/G/T (decimal). $1Gi = 1024^3 B$.
- **Párovanie s PVC:** PVC dostane PV, ktoré má `capacity.storage >= PVC requests.storage`.

`spec.accessModes`

- **Povinné.** Zoznam módov, v ktorých sa môže PV pripojiť.
- **Hodnoty:**
 - ReadWriteOnce (RWO) — jeden node read-write. Tu zvolené, lebo hostPath nepodporuje viac.
 - ReadOnlyMany (ROX) — viac nodov read-only.
 - ReadWriteMany (RWX) — viac nodov read-write (potrebuje NFS / CephFS / atď.).
 - ReadWriteOncePod (RWOP) — len 1 Pod read-write (1.22+).
- **Pre toto zadanie:** RWO stačí, lebo replicas: 1 a jeden Pod.

`spec.hostPath.path: "/mnt/data/exam-horvath"`

- **Povinné v rámci hostPath driveru.**
- **Význam:** PV mapuje *priečinok na konkrétnom node* (v minikube je to vnútro VM/kontajnera).
- **Pozor:** hostPath je **dev-only** — v produkcii sa NEPOUŽÍVA (nemá sieťovú dostupnosť, nemá replication, padá pri zmene node-u).
- **Pre minikube:** treba minikube ssh a vytvoriť priečinok `sudo mkdir -p /mnt/data/exam-horvath`. Inak Pod môže fail-núť na *MountVolume*.

Implicitné default hodnoty (ktoré tu *nie sú*, ale dobré vedieť)

- `spec.persistentVolumeReclaimPolicy`: Retain (default pre manuálne PV) — po zmazaní PVC sa dáta v PV nezmažú.
 - Iné: Delete (zmaže aj PV), Recycle (deprecated).

Čo by sa dalo pridať

- `spec.volumeMode`: Filesystem (default) alebo Block.
- `spec.nodeAffinity` — viaže PV na konkrétny node (pri multi-node klasteri).
- `spec.persistentVolumeReclaimPolicy`: Retain — explicitne.

3b. k8s/pvc.yaml — PersistentVolumeClaim

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: exam-pvc
  namespace: exam-horvath
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Pole-po-poli

apiVersion: v1 + kind: PersistentVolumeClaim

- **Povinné. Core objekt.**

metadata.name: exam-pvc

- **Povinné.** Referencované z Deployment cez persistentVolumeClaim.claimName: exam-pvc.

metadata.namespace: exam-horvath

- **Dôležité** — PVC **musí byť v rovnakom namespace** ako Pod, ktorý ho používa.

spec.storageClassName: manual

- **Musí sa zhodovať** s storageClassName v PV (exam-pv), inak sa neviažu (Pending PVC).

spec.accessModes: [ReadWriteOnce]

- **Musí byť podmnožinou** access módov PV. [RW0] $\subseteq$ [RW0] ✓.

spec.resources.requests.storage: 1Gi

- **Povinné.** Koľko miesta PVC žiada.
- **Pravidlo binding-u:** K8s nájde najmenšie PV, ktoré spĺňa **všetky** požiadavky (storageClassName, accessModes, capacity $\geq$ requests, žiadny selector mismatch). Tu máme jediné PV exam-pv (1Gi) → naviaže sa naň.

Workflow PV ↔ PVC

1. `kubectl apply -f pv.yaml` → PV vznikne v stave Available.
2. `kubectl apply -f pvc.yaml` → controller hľadá vhodné PV.
3. Po naviazaní: PV má Status: Bound, PVC má Status: Bound a Volume: exam-pv.
4. Deployment cez claimName namountuje obsah PVC do Podu.

Časté zlyhania

- PVC zostane Pending → najčastejšie:
 - storageClassName sa nezhoduje.
 - accessModes PV nepodporuje.
 - requests.storage > capacity.storage PV.
 - V minikube neexistuje /mnt/data/exam-horvath priečinok.

4 + 5. k8s/deployment-upload-init.yaml — Deployment + Init-Container

Najobsiahlejší súbor — obsahuje **5. Deployment** aj **4. InitContainer** v jednej template (lebo Init-Container je súčasťou Pod template, nie samostatný objekt).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: exam-app
  namespace: exam-horvath
spec:
  replicas: 1
  selector:
    matchLabels:
      app: exam-app
  template:
    metadata:
      labels:
        app: exam-app
    spec:
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: exam-pvc
        - name: cfg
          configMap:
            name: app-config
            items:
              - key: WELCOME_TEXT
                path: welcome.txt
              - key: SAMPLE_CSV_1
                path: sample1.csv
              - key: SAMPLE_CSV_2
                path: sample2.csv

      initContainers:
        - name: init-data
          image: busybox:1.36
          command: ["sh", "-c"]
          args:
            - |
              set -e
              mkdir -p /data/input /data/output
              cp /cfg/welcome.txt /data/welcome.txt
              if [ -z "$(ls -A /data/input 2>/dev/null)" ]; then
                cp /cfg/sample1.csv /data/input/sample1.csv
                cp /cfg/sample2.csv /data/input/sample2.csv
              fi
              chown -R 33:33 /data
      volumeMounts:
        - name: data
          mountPath: /data
        - name: cfg
          mountPath: /cfg
```

```

containers:
  - name: exam-app
    image: gitdockeracc/exam-app:latest
    ports:
      - containerPort: 80
    envFrom:
      - configMapRef:
          name: app-config
    volumeMounts:
      - name: data
        mountPath: /var/www/html/data
    readinessProbe:
      httpGet:
        path: /ready.php
        port: 80
      initialDelaySeconds: 3
      periodSeconds: 5
    livenessProbe:
      httpGet:
        path: /health.php
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 10

```

Top-level polia

apiVersion: apps/v1

- **Povinné. POZOR:** Deployment **nie je v v1** — patrí do **API group apps** od K8s 1.9. Staršie apps/v1beta1 / extensions/v1beta1 sú odstránené.

kind: Deployment

- **Povinné.**

metadata.name: exam-app

- **Povinné.** Tento názov sa premieta do `kubectl rollout status deployment/exam-app`.

metadata.namespace: exam-horvath

- Rovnaký namespace ako PVC + ConfigMap.

spec: Deployment-u

spec.replicas: 1

- **Povinné? NIE (default = 1)**, ale explicitne uvedené.
- **Tu je 1** lebo PVC má ReadWriteOnce — viac replík by sa nedalo namountovať na rovnaký PV.

spec.selector.matchLabels.app: exam-app

- **Povinné a MUSÍ sa zhodovať** s `template.metadata.labels`. Ak nie, Deployment ohlási *“selector does not match template labels”* a odmietne sa vytvoriť.
- **Trvalý field:** selector je immutable po vytvorení.

`spec.template:` — **Pod template**

- **Povinné.** Toto je definícia Podu, ktorý Deployment vytvorí.

`template.metadata.labels.app:` **exam-app**

- **Povinné.** Musí sa zhodovať so selectorom (vyššie).
- Tieto labely používa **Service** na nájdenie Podov (`service.yaml.spec.selector.app: exam-app`).

`template.spec.volumes:` — **Pod-level volumes**

Volume data (PVC)

```
- name: data
  persistentVolumeClaim:
    claimName: exam-pvc
```

- `name: data` — lokálne meno volume v rámci Podu.
- `persistentVolumeClaim.claimName: exam-pvc` — odkaz na PVC (musí byť v rovnakom namespace).

Volume cfg (ConfigMap projected)

```
- name: cfg
  configMap:
    name: app-config
    items:
      - key: WELCOME_TEXT
        path: welcome.txt
      - key: SAMPLE_CSV_1
        path: sample1.csv
      - key: SAMPLE_CSV_2
        path: sample2.csv
```

- `configMap.name: app-config` — názov ConfigMapu (musí existovať v rovnakom namespace).
- `items:` — **selektívne** premapovanie kľúčov na názvy súborov. Bez `items` by sa všetky kľúče namontovali ako súbory rovnakého mena.
 - `key:` kľúč v ConfigMape.
 - `path:` meno súboru v mountovanom priečinku.
- **Read-only:** ConfigMap volumes sú vždy `readOnly`. InitContainer preto **kopíruje** obsah (`cp /cfg/... /data/...`), nepíše priamo do `/cfg`.

`template.spec.initContainers:` — **InitContainer**

InitContainer **beží PRED** hlavnými kontajnermi, **musí úspešne skončiť (exit 0)**, až potom sa štartuje containers. Pri zlyhaní sa Pod restartuje (`restartPolicy: Always` default).

`- name: init-data`

- **Povinné.** Unikátne v rámci Podu.

`image: busybox:1.36`

- **Povinné.** Minimalistický image (~5 MB) s `sh`, `cp`, `mkdir`, `chown`, `ls`.
- **Tag 1.36:** explicit verzia — vyhneš sa “latest” lottery.

command: ["sh", "-c"]

- **Voliteľné** — prepisuje ENTRYPOINT image-u.
- ["sh", "-c"] = "spusti shell, ktorý interpretuje string ako command".
- **Bez tohto** by sa spustil default entrypoint busyboxu (/bin/sh) bez argumentu.

args: (s | literal block)

- **Voliteľné** — prepisuje CMD.
- Pripája sa ako argumenty za command. Výsledný príkaz: `sh -c "<celý shell skript>".`

Riadky shell skriptu InitContaineru

Riadok	Význam
<code>set -e</code>	Pri prvej chybe okamžite skončí (exit code $\neq 0$).
<code>mkdir -p /data/input /data/output</code>	Vytvorí input/ a output/ ak ešte nie sú (-p = nepadne ak existujú).
<code>cp /cfg/welcome.txt /data/welcome.txt</code>	Skopíruje welcome súbor z ConfigMap volume do PVC.
<code>if [-z "\$(ls -A /data/input)"];</code> <code>then ...</code>	Iba ak je input/ prázdny, naseje sample CSV (idempotencia — pri restarte Podu nepretečie zmenené dáta).
<code>cp /cfg/sample1.csv /data/input/sample1.csv</code>	Sample CSV ako vstupné dáta.
<code>chown -R 33:33 /data</code>	UID/GID 33 = www-data v php:8.2-apache image-i. Bez tohto Apache nemôže písať.

volumeMounts:

```
- name: data
  mountPath: /data
- name: cfg
  mountPath: /cfg
```

- Namountuje PVC volume na /data a ConfigMap volume na /cfg v rámci tohto InitContaineru.
- **Mountpathy sú nezávislé** od hlavného containeru (ten má PVC na /var/www/html/data).

template.spec.containers: — hlavný kontajner

- **name:** exam-app

- Unikátne v rámci Podu.

image: gitdockeracc/exam-app:latest

- PHP + Apache image z Docker Hubu (php:8.2-apache so skopírovaným src/).
- **:latest tag:** pohodlné, ale neodporúčané pre produkciu (Pod môže ťahať starú verziu z cache).
- **imagePullPolicy:** default je Always pre :latest a IfNotPresent pre konkrétne tagy.

ports.containerPort: 80

- **Iba informatívne** — neotvára port, len informuje K8s a ostatné nástroje. Apache počúva na 80 už z image-u.

`envFrom.configMapRef.name: app-config`

- Vloží **každý** klíč z ConfigMapu `app-config` ako env var s rovnakým menom.
- Aplikácia tak má `APP_NAME`, `INPUT_DIR`, `OUTPUT_DIR`, ... cez `getenv()`.
- **Alternatíva:** `env: [- name: APP_NAME, valueFrom: configMapKeyRef: ...]` — selektívnejšie, ale viac riadkov.

`volumeMounts:`

```
- name: data
  mountPath: /var/www/html/data
```

- PVC sa mountuje **dovnútra Apache document rootu** (`/var/www/html/`) ako podpriechinok `data/`.
- Vďaka tomu `index.php` vidí `data/input/` a `data/output/`.

`readinessProbe: (BONUS)`

```
httpGet:
  path: /ready.php
  port: 80
initialDelaySeconds: 3
periodSeconds: 5
```

- **Účel:** Service nezačne posilať traffic na Pod, kým `readinessProbe` nevráti 200.
- `path: /ready.php` — náš PHP endpoint kontroluje, či je `/var/www/html/data` writable.
- `initialDelaySeconds: 3` — počká 3s po štarte containera než začne probovať.
- `periodSeconds: 5` — opakuje každých 5s.
- **Default thresholds:** `failureThreshold: 3`, `successThreshold: 1`, `timeoutSeconds: 1`.

`livenessProbe: (BONUS)`

```
httpGet:
  path: /health.php
  port: 80
initialDelaySeconds: 10
periodSeconds: 10
```

- **Účel:** Ak probe zlyhá `failureThreshold` krát, kubelet **kontajner zabije a restartuje**.
- Náš `health.php` len vráti `OK` — overuje, že Apache + PHP-FPM nezamrzli.

Polia, ktoré tu nie sú (default / mohli by byť)

- `resources.requests / limits` — CPU/RAM rezervácie.
- `securityContext.runAsUser: 33` — eliminuje potrebu `chown` v `InitContaineri`.
- `env` — explicit env override.
- `imagePullSecrets` — pre privátne registre.

6. k8s/service.yaml — Service NodePort

```
apiVersion: v1
kind: Service
metadata:
  name: exam-svc
  namespace: exam-horvath
spec:
  selector:
    app: exam-app
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

Pole-po-poli

apiVersion: v1 + kind: Service

- **Povinné.** Service je core objekt.

metadata.name: exam-svc

- **Povinné.** Bude dostupný cez DNS ako `exam-svc.exam-horvath.svc.cluster.local`.

metadata.namespace: exam-horvath

- **Service musí byť v rovnakom namespace ako Pody, ktoré selektuje** (selector je obmedzený na namespace).

spec.selector.app: exam-app

- **Povinné** (pri ClusterIP/NodePort/LoadBalancer).
- **Musí sa zhodovať** s `template.metadata.labels` v Deployment (`app: exam-app`).
- Ak labely nesedia, Service nemá žiadne endpointy → `kubectl get endpoints exam-svc` ukáže `<none>`.

spec.type: NodePort

- **Voliteľné — default ClusterIP.**
- **Hodnoty:**
 - ClusterIP — len interný IP v klastru.
 - NodePort — navyše otvorí port **na každom node-e** (default rozsah 30000–32767).
 - LoadBalancer — navyše vytvorí cloud LB (AWS ELB, GCP LB, ...).
 - ExternalName — DNS CNAME, žiadny proxying.
- **Zadanie chce explicitne NodePort. ✓.**

spec.ports: — zoznam portov

port: 80

- **Povinné.** Port, na ktorom je Service dostupný **vnútri klastra** (cluster-IP).
- Iný Pod by k tomuto pristúpil cez `http://exam-svc:80`.

targetPort: 80

- **Voliteľné — default = port.**
- Port v rámci Podu, kam Service redirekuje traffic. Tu Apache počúva na 80.
- Môže byť aj name (napr. targetPort: http), ak je v containeri ports.name: http.

nodePort: 30080

- **Voliteľné — ak vynecháš, K8s ti automaticky priradí port z 30000-32767.**
- Tu zámerne fixujem na 30080, aby URL bola stabilná: http://<node-IP>:30080.
- **Pravidlo:** musí byť v rozsahu 30000-32767 (default --service-node-port-range).

Default polia (neprítomné, ale dobre vedieť)

- spec.sessionAffinity: None (default) alebo ClientIP.
- spec.externalTrafficPolicy: Cluster (default) alebo Local — Local zachová zdrojovú IP, ale traffic ide len na nody, kde beží Pod.
- spec.protocol: TCP (default), iné: UDP, SCTP.

Ako sa volá služba zvonku

- **Minikube:** minikube service exam-svc -n exam-horvath (otvorí v prehliadači a vypíše URL).
- **Cez tunel:** minikube tunnel + IP nodu z minikube ip.
- **Priamo:** http://\$(minikube ip):30080.
- **Lokálne mapovanie:** kubectl port-forward -n exam-horvath svc/exam-svc 30080:80
→ http://localhost:30080.

7. Porty a sieťovanie — kompletný rozbor

V projekte sa vyskytuje port 80 (interný HTTP) a 30080 (NodePort) na viacerých miestach. Toto je kompletný tok packetu od klienta k Apache procesu.

7.1 Tabuľka všetkých portov

Port	Kde je definovaný	Typ	Účel
80	php:8.2-apache image (default Apache Listen 80)	container	Apache HTTP server počúva tu vnútri kontajnera.
80	deployment-upload-init.yaml ports.containerPort: 80	dokumentácia	farmaceutické pole — neotvára port, len deklaruje, kde Apache počúva.
80	service.yaml spec.ports.targetPort: 80	service→pod	Service forwarduje traffic na tento port v Pode .
80	service.yaml spec.ports.port: 80	cluster IP	Port Service-u vnútri klastra (exam-svc:80).
30080	service.yaml spec.ports.nodePort: 30080	node→service	Otvorený na každom node-e , prístupný zvonku klastra.
80	readinessProbe.httpGet.port: 80 (Deployment)	probe→pod	Kubelet posíla GET /ready.php priamo na Pod IP:80.
80	livenessProbe.httpGet.port: 80 (Deployment)	probe→pod	Kubelet posíla GET /health.php priamo na Pod IP:80.

7.2 Tok packetu (request) — krok po kroku

```
[Klient]                                     http://localhost:30080/
|
| v minikube: localhost:30080 cez minikube routing alebo
| kubectrl port-forward, alebo $(minikube ip):30080
v
[Node] port 30080 (otvorený kube-proxy-om)
|
| kube-proxy (iptables/ipvs/nftables) DNAT-uje paket na ClusterIP:80
v
[Service exam-svc] ClusterIP:80
|
| load balancing medzi endpointami (tu je len 1 Pod)
v
[Pod] PodIP:80 (= targetPort)
|
| kontajner sieťovo žije na rovnakej IP ako Pod (default)
v
[Container exam-app] 127.0.0.1:80
|
| Apache `Listen 80` (z php:8.2-apache image-u)
v
[Apache] -> /var/www/html/index.php
```

7.3 Prečo sú porty rovnaké?

V tomto projekte sú containerPort, targetPort a port všetky 80 — to zjednodušuje debug, ale K8s nepožaduje ich zhodu. Ako sa to dá rozdeliť:

- containerPort: 8080 (Apache zmenený na Listen 8080),

- targetPort: 8080 (Service vie, že má volať Pod na :8080),
- port: 80 (zvonku Service ostane na :80),
- nodePort: 30080 (zvonku z host-u stále 30080).

7.4 Probes — port flow

Probes **nejdú cez Service** — kubelet volá Pod IP priamo:

```
[kubelet (na node-e)]
|
|   každých `periodSeconds` (5s / 10s)
|   HTTP GET PodIP:80/ready.php
|   HTTP GET PodIP:80/health.php
v
[Apache] -> ready.php / health.php
|
v
returncode 200 -> probe OK
returncode 5xx -> probe FAIL
```

Preto je port: 80 v probes **port v Pode** (ten istý ako containerPort), NIE port Service.

7.5 Rozsahy a obmedzenia

Vlastnosť	Hodnota / pravidlo
NodePort range (default)	30000–32767 (kube-apiserver --service-node-port-range)
Privileged port (< 1024) v Pode	Vyžaduje CAP_NET_BIND_SERVICE alebo securityContext.runAsUser: 0. php:8.2-apache má root entrypoint preto môže bindovať 80.
Konflikt NodePort-u	Dvaja Service nemôžu mať rovnaký nodePort — apply zlyhá s port is already allocated.
Auto-priradený NodePort protocol	Ak nodePort: vynecháš, K8s ti vyberie voľný z rozsahu. TCP (default), UDP, SCTP.

7.6 Ako sa dostať na port zvonku

Metóda	Príkaz	URL
minikube service	minikube service exam-svc -n exam-horvath	(otvorí v prehliadači)
Priama IP node-u	minikube ip (zistí IP) + port 30080	http://192.168.49.2:30080
kubectl port-forward (svc)	kubectl port-forward -n exam-horvath svc/exam-svc 30080:80	http://localhost:30080
kubectl port-forward (pod)	kubectl port-forward -n exam-horvath pod/<pod> 8080:80	http://localhost:8080
Vnútri klastra (iný Pod)	curl http://exam-svc.exam- horvath.svc.cluster.local	(DNS Service-u)

7.7 Probes — overenie v praxi

```
# stav probe-ov v Pode (Conditions: Ready True/False, Restarts)
kubectl describe pod -n exam-horvath -l app=exam-app
```

```
# manuálne otestuj endpointy
kubectl port-forward -n exam-horvath svc/exam-svc 30080:80
curl -v http://localhost:30080/ready.php      # očakávané: 200 READY
curl -v http://localhost:30080/health.php     # očakávané: 200 OK

# počet restartov (livenessProbe failures = restart)
kubectl get pods -n exam-horvath
```

8. URL, cesty, endpointy

URL / cesta	Kde je definovaná	Účel
http://localhost:30080/	service.yaml (nodePort) + minikube	Hlavné UI aplikácie (index.php)
http://localhost:30080/ready/	ready.php + Deployment readinessProbe	Pripravenosť Podu (kontrola write na /data)
http://localhost:30080/health/	health.php + Deployment livenessProbe	Živosť kontajnera
exam-svc.exam-horvath.svc.cluster.local	implicit DNS pre Service	Interná DNS adresa služby
gitdockeracc/exam-app:latest	image: v Deployment	Docker Hub repository — odkiaľ sa ťahá image
/var/www/html/	Apache document root	Document root v hlavnom kontajneri
/var/www/html/data	volumeMounts.mountPath	Mountpoint PVC v hlavnom kontajneri
/var/www/html/data/input	ConfigMap INPUT_DIR	Vstupné CSV (vytvára InitContainer)
/var/www/html/data/output	ConfigMap OUTPUT_DIR	Spracované CSV
/var/www/html/data/welcome	InitContainer cp /cfg/welcome.txt /data/welcome.txt	Welcome text z ConfigMapu
/data	InitContainer volumeMounts.mountPath	Mountpoint PVC v init kontajneri
/cfg	InitContainer volumeMounts.mountPath	Mountpoint ConfigMap volume v init kontajneri
/mnt/data/exam-horvath	pv.yaml hostPath.path	Reálna cesta na node-e (minikube VM)

9. Příkazy — kompletný cheat sheet

9.1 Predpoklady (jednorazovo)

`minikube start`

- Spustí lokální single-node klaster.
- Před prvním apply: ak používáš hostPath, vytvoř přechýnky:

```
minikube ssh
sudo mkdir -p /mnt/data/exam-horvath
sudo chmod 777 /mnt/data/exam-horvath
exit
```

9.2 Nasadenie

`kubectl apply -f k8s/namespace.yaml`

- Vytvorí namespace. **Musí být první** — všechno ostatní je v něm.

`kubectl apply -f k8s/`

- Aplikuje **všetky** YAML v přechýnku naraz.
- `kubectl` zoradí podle závislostí *částečně* — namespace jde vždy první.

Postupné nasadenie (manuálně, v správném pořadí)

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/pv.yaml
kubectl apply -f k8s/pvc.yaml
kubectl apply -f k8s/deployment-upload-init.yaml
kubectl apply -f k8s/service.yaml
```

9.3 Diagnostika

`kubectl get all -n exam-horvath`

- Přehled všech namespaced objektů (deployment, replicaset, pod, service).
- **Nezahrňa:** ConfigMap, PVC, Secret — tie treba samostatne.

`kubectl get pods -n exam-horvath`

- Status Podu. Stĺpec READY = running/total kontajnery (init nepočíta po skončení).
- Stĺpec STATUS: Init:0/1 → InitContainer beží, Running → hlavný kontajner beží.

`kubectl describe pod <pod-name> -n exam-horvath`

- Plné info: events, conditions, mount errors, probe failures.
- **Prvý nástroj pri akejkoľvek chybe.**

`kubectl logs <pod-name> -n exam-horvath`

- Stdout/stderr hlavného kontajnera.

kubectl logs <pod-name> -c init-data -n exam-horvath

- -c = ktorý kontajner. Bez tohto by default vybral hlavný a InitContainer logy by si nevidel.

kubectl exec -it <pod-name> -n exam-horvath -- sh

- Interaktívny shell vnútri kontajnera.
- Vhodné na kontrolu mountov: `ls /var/www/html/data/input`.

kubectl get pv,pvc -n exam-horvath

- Status väzby PV ↔ PVC. Obidve musia byť Bound.

kubectl get cm -n exam-horvath

- Zoznam ConfigMapov. `kubectl describe cm app-config -n exam-horvath` ukáže obsah.

kubectl get svc -n exam-horvath

- Service + priradené NodePort číslo.

kubectl get endpoints exam-svc -n exam-horvath

- Či má Service IP adresy Podov. Prázdne = label selector neseďí.

9.4 Testovanie

minikube service exam-svc -n exam-horvath

- Otvorí URL v prehliadači.

kubectl port-forward -n exam-horvath svc/exam-svc 30080:80

- Mapovanie localhost:30080 → Service port 80.
- Beží v popredí (Ctrl+C zastaví).

curl http://localhost:30080/

- HTML obsah index.php.

curl http://localhost:30080/ready.php

- Vráti READY (alebo NOT READY).

curl http://localhost:30080/health.php

- Vráti OK.

9.5 Update / re-deploy

kubectl rollout restart deployment/exam-app -n exam-horvath

- Reštartuje všetky Pody Deploymentu (vznikne nový ReplicaSet).
- Použiteľné po zmene ConfigMapu — env vars sa **Nenačítajú znova bez restartu Podu**.

```
kubectl rollout status deployment/exam-app -n exam-horvath
```

- Sleduje progres rolloutu.

```
kubectl set image deployment/exam-app exam-app=gitdockeracc/exam-app:v2 -n exam-horvath
```

- Zmena image-u bez editácie YAMLu.

9.6 Cleanup

```
kubectl delete -f k8s/
```

- Zmaže všetko, čo bolo applied z k8s/.

```
kubectl delete namespace exam-horvath
```

- Zmaže celý namespace **a všetko v ňom** (deployment, pods, svc, configmap, pvc, ...).
- **Pozor:** PV je cluster-scoped → ostane (so stavom Released).

```
kubectl delete pv exam-pv
```

- Po zmazaní namespace ručne zmaž aj PV.

10. Tok dát — kompletný diagram

- 1) `kubectl apply`
 - |
 - v
- 2) API server vytvorí objekty
namespace -> configmap -> pv -> pvc (bound k pv) -> deployment -> service
 - |
 - v
- 3) Deployment vytvorí Pod
 - |
 - v
- 4) Pod startuje InitContainer (init-data)
 - | - mountuje volume "data" (PVC) na /data
 - | - mountuje volume "cfg" (ConfigMap) na /cfg
 - | - mkdir -p /data/input /data/output
 - | - cp /cfg/welcome.txt /data/welcome.txt
 - | - ak je /data/input prázdny: cp /cfg/sample*.csv /data/input/
 - | - chown -R 33:33 /data
 - | - exit 0
 - |
 - v
- 5) Pod startuje hlavný container (exam-app)
 - | - env vars z ConfigMapu (envFrom)
 - | - mount PVC -> /var/www/html/data
 - | - Apache + PHP počúva na :80
 - |
 - v
- 6) readinessProbe (GET /ready.php) -> READY
 - |
 - v
- 7) Service (NodePort :30080) routuje traffic na Pod:80
 - |
 - v
- 8) Klient: `http://localhost:30080/`
 - |
 - v
- 9) `index.php`:
 - prečíta CSV z /var/www/html/data/input
 - spracuje (pridá hlavičku "# PROCESSED: ...")
 - zapíše do /var/www/html/data/output
 - vráti HTML s welcome textom + tabuľkami CSV

11. Checklist — pred odovzdaním

- ☐ `kubectl get all,cm,pvc,pv -n exam-horvath` ukazuje všetko v poriadku.
- ☐ Pod má status `Running` a `1/1 Ready`.
- ☐ `kubectl logs <pod> -c init-data -n exam-horvath` ukazuje úspešné dokončenie initu.
- ☐ `http://localhost:30080/` zobrazí `welcome text` + 2 CSV tabuľky.
- ☐ V `output/` (`kubectl exec ... -- ls /var/www/html/data/output`) sú spracované CSV súbory.
- ☐ `README.md` existuje a obsahuje: popis, `kubectl apply -f`, testovaciu URL, vysvetlenie komponentov.
- ☐ **Bonus doložený:** `readinessProbe` + `livenessProbe` (kontrola `kubectl describe pod`).